

Module 17: Google Kubernetes Engine, Stateful Workloads, Microservices DNS, Ingress, ConfigMap, Secrets, and CI/CD

Questions

Q1. What is Google Kubernetes Engine, and why would a team choose it?

Answer: Google Kubernetes Engine is a managed Kubernetes service provided by Google Cloud. It handles the control plane, including the API server, scheduler, etcd, and controller manager, while users manage the worker nodes and workloads. Teams choose GKE because it reduces operational burden compared to self-managed Kubernetes, integrates with Google Cloud services such as Cloud Storage, Cloud SQL, and IAM, and offers features like automatic node repair, cluster autoscaling, and managed upgrades. For backend and DevOps engineers, GKE provides a production-ready Kubernetes platform with built-in networking, security, and observability integrations.

Q2. What is the difference between GKE Standard and GKE Autopilot?

Answer: GKE Standard gives you full control over node pools, machine types, scaling, and cluster configuration. You pay for the nodes and manage capacity planning. GKE Autopilot abstracts node management away; Google automatically provisions, scales, and secures nodes based on the Pods you deploy. Autopilot charges per Pod rather than per provisioned node and enforces best practices such as resource requests, restricted security settings, and allowed container images. Standard is better for workloads that need custom node configuration, GPUs, or privileged containers. Autopilot is better for teams that want to focus on applications rather than infrastructure.

Q3. How do you create a GKE cluster using gcloud CLI?

Answer: First, set the project and region with `gcloud config set project <project-id>` and `gcloud config set compute/region <region>`. Then create the cluster with `gcloud container clusters create <cluster-name> --zone <zone> --num-nodes <count>`. For Autopilot, use `gcloud container clusters create-auto <cluster-name> --region <region>`. After creation, fetch credentials with `gcloud container clusters`

`get-credentials <cluster-name> --zone <zone>` so kubectl can interact with the cluster. You can also specify machine type, network, subnet, release channel, and node labels as flags.

Q4. What are common gcloud commands used to manage GKE clusters?

Answer: Common commands include `gcloud container clusters create` to create a cluster, `gcloud container clusters list` to list clusters, `gcloud container clusters describe` to inspect details, `gcloud container clusters get-credentials` to configure kubectl, `gcloud container clusters update` to modify cluster settings, and `gcloud container clusters delete` to remove a cluster. For node pools, `gcloud container node-pools list`, `create`, `delete`, and `update` are used. For workloads, you usually switch to kubectl commands after fetching credentials. These commands are essential for DevOps automation and local cluster management.

Q5. What is Workload Identity in GKE, and why is it important?

Answer: Workload Identity allows a Kubernetes service account to impersonate a Google Cloud IAM service account without storing long-lived credentials inside Pods. A Kubernetes service account is mapped to a Google service account through an annotation and IAM policy binding. Applications can then use Google Cloud client libraries to access services like Cloud Storage, Pub/Sub, and Secret Manager with fine-grained IAM permissions. It is important because it eliminates the need to mount JSON key files, which are a security risk if leaked. It follows the principle of least privilege and rotates tokens automatically.

Q6. How do you set up a database in GKE using a StatefulSet?

Answer: A StatefulSet is used because databases require stable network identity and persistent storage. The StatefulSet defines a `volumeClaimTemplate` so each replica gets its own `PersistentVolumeClaim` backed by a Google Cloud persistent disk through a `StorageClass`. A headless `Service` provides stable DNS names for each Pod, such as `db-0.db.default.svc.cluster.local`. `ConfigMaps` can separate primary and replica configuration, and `Secrets` store credentials. `Init` containers can perform bootstrapping tasks like setting server IDs or cloning data from a peer. `Readiness` and `liveness` probes are configured to verify the database is healthy before traffic is sent or before Kubernetes restarts the Pod.

Q7. Why should a StatefulSet use a headless Service for a database cluster?

Answer: A headless Service has `clusterIP: None` and does not load balance. Instead, it creates DNS records for each individual Pod. For a database cluster, members need to discover and communicate with specific peers by stable name. For example, a MySQL InnoDB cluster can use `mysql-0.mysql`, `mysql-1.mysql`, and `mysql-2.mysql` as stable hostnames. A regular ClusterIP Service would return one virtual IP and distribute traffic, which breaks protocols that require direct peer connections. Headless Services are therefore essential for protocols and clustering mechanisms that rely on per-instance identity.

Q8. How do Kubernetes microservices discover each other using DNS?

Answer: Kubernetes runs an internal DNS service, usually CoreDNS, inside the cluster. Each Service receives a DNS name in the format `<service-name>.<namespace>.svc.cluster.local`. Pods can resolve this name to the Service's ClusterIP, and traffic is load-balanced across backend Pods. Headless Services return the individual Pod IPs. For cross-namespace communication, the full DNS name including the namespace is used. DNS-based discovery allows microservices to find dependencies without hard-coding IP addresses, and it works automatically as Pods are created, deleted, or rescheduled.

Q9. What is the difference between a ClusterIP, NodePort, and LoadBalancer Service?

Answer: ClusterIP exposes the Service only inside the cluster on an internal IP. It is the default and is used for internal microservice communication. NodePort exposes the Service on a static port of every node, allowing external traffic to reach the Service through any node's IP. LoadBalancer provisions a cloud provider load balancer with a public IP and forwards traffic to the Service. In GKE, LoadBalancer Services create a Google Cloud network load balancer. For HTTP applications, Ingress is usually preferred over LoadBalancer because it supports path-based routing, TLS termination, and cost sharing.

Q10. What is a Kubernetes Ingress, and how is it used in GKE?

Answer: An Ingress is a Kubernetes resource that exposes HTTP and HTTPS routes from outside the cluster to Services inside the cluster. In GKE, an Ingress resource is fulfilled by GKE Ingress Controller, which provisions a Google Cloud HTTP load balancer. Ingress supports host-based and path-based

routing, TLS termination using Kubernetes Secrets, and backend health checks. For backend developers, Ingress is the entry point for external API traffic. For DevOps engineers, it centralizes external access, SSL management, and routing rules without creating multiple LoadBalancer Services.

Q11. How do you reserve a static IP for a GKE Ingress?

Answer: You create a global static IP in GCP with `gcloud compute addresses create <name> --global`, then reference it in the Ingress annotation `networking.gke.io/static-ip: <name>`. The IP must be in the same region as the load balancer for regional Ingress, or global for global Ingress. This ensures the external IP does not change when the Ingress is recreated. For production APIs and websites, a static IP is important because DNS records can point to it reliably.

Q12. What is the difference between a ConfigMap and a Secret?

Answer: ConfigMaps store non-sensitive configuration data such as environment variables, command-line arguments, and configuration files. Secrets store sensitive data such as passwords, tokens, and TLS certificates. Both can be mounted as volumes or injected as environment variables. Values in Secrets are base64-encoded but not encrypted by default unless etcd encryption at rest is enabled. Secrets should be used for sensitive data, but they should still be protected with RBAC and integrated with external secret managers when possible. ConfigMaps can be updated more freely and are safe to store in version control if they contain no secrets.

Q13. How do you manage environment-specific configuration in GKE?

Answer: I use separate namespaces for each environment such as dev, staging, and prod. Each namespace contains its own ConfigMaps and Secrets. The Deployment references these resources, so the same container image can run in any environment with the correct configuration. For managing many environments, I use Helm or Kustomize to generate manifests per environment from a common base. Sensitive values are stored in Google Secret Manager and injected by an external secrets operator or Workload Identity, rather than being committed to Git. This separation keeps images immutable and configuration environment-specific.

Q14. How do you securely store and use secrets in GKE?

Answer: The best approach is to use Workload Identity to allow Pods to fetch secrets directly from Google Secret Manager at runtime. Alternatively, I use an external secrets operator such as External Secrets Operator or Sealed Secrets for GitOps workflows. Native Kubernetes Secrets can be used for simple cases, but they should be encrypted at rest in etcd, access should be restricted with RBAC, and

they should be mounted as files rather than environment variables when possible. I never store secrets in container images, ConfigMaps, or Git repositories in plain text. Regular rotation and audit logging are also important.

Q15. What is the typical CI/CD flow for deploying to GKE using GitHub Actions?

Answer: The CI/CD flow usually has two stages. The continuous integration stage runs on every pull request or push to the main branch. It builds the application, runs tests, performs code quality checks, and builds a Docker image. The image is tagged with the Git commit SHA and pushed to Artifact Registry. The continuous delivery stage is triggered after a successful build. It updates the Kubernetes manifest, typically with the new image tag, and applies it to the cluster. In a GitOps approach, GitHub Actions only builds and pushes the image, while a tool like ArgoCD detects the manifest change and syncs the cluster. This separates image building from deployment.

Q16. What is GitOps, and how does it apply to GKE deployments?

Answer: GitOps is an operational model where Git is the single source of truth for infrastructure and application configurations. In GKE, Kubernetes manifests or Helm charts are stored in a Git repository. A GitOps controller such as ArgoCD or Flux watches the repository and continuously reconciles the cluster state with the desired state in Git. When a developer merges a change, the controller applies it. GitOps provides version control, audit trails, easy rollbacks by reverting a commit, and drift detection. It is popular for GKE deployments because it makes deployment history transparent and automates complex multi-environment management.

Q17. What is the difference between push-based and pull-based CI/CD?

Answer: In push-based CI/CD, a pipeline such as GitHub Actions builds the artifact and then pushes configuration changes directly to the cluster using kubectl. The pipeline controls when deployments happen. In pull-based CI/CD, the pipeline builds and pushes the image, then updates a Git repository. A controller running in the cluster such as ArgoCD pulls the desired state from Git and applies it. Pull-based deployment is more secure because the cluster does not need long-lived external credentials to be modified, and it supports automatic drift correction. Push-based deployment is simpler but can lead to configuration drift if manual cluster changes occur.

Q18. What are common deployment strategies for GKE?

Answer: Common strategies include rolling updates, blue-green deployments, canary deployments, and recreates. Rolling updates replace Pods gradually using Deployment strategy settings. Blue-green runs two identical environments and switches traffic between them. Canary deploys a new version to a small subset of users and gradually increases traffic while monitoring metrics. Recreate stops old Pods before starting new ones, causing downtime and is rarely used for production services. GKE supports these through Kubernetes Deployments, Ingress traffic splitting, Istio, or Argo Rollouts.

Q19. How do you implement a canary deployment in GKE?

Answer: A canary deployment can be implemented with a Service mesh such as Istio or with GKE Ingress traffic splitting. With Istio, you define a VirtualService that routes a small percentage of traffic to the canary version based on labels. With GKE Ingress, you create a BackendConfig with traffic splitting weights across two Services. You deploy the canary version as a separate Deployment with a small replica count, route 5 or 10 percent of traffic to it, monitor error rates and latency, and gradually increase the percentage if healthy. Automated rollback triggers if metrics exceed thresholds.

Q20. How do you monitor and log GKE workloads?

Answer: GKE integrates with Google Cloud Logging and Cloud Monitoring, which collect container logs and metrics automatically. I configure structured logging so logs can be filtered and analyzed easily. For metrics, I use Cloud Monitoring dashboards and alert policies for CPU, memory, disk, and custom application metrics. I also deploy Prometheus and Grafana for Kubernetes-native metrics such as pod resource usage, node health, and HPA behavior. For distributed systems, I add tracing with Cloud Trace or OpenTelemetry. Logs, metrics, and traces together provide full observability.

Q21. What is a StorageClass in GKE, and how do you use it?

Answer: A StorageClass defines the type of storage provisioned for PersistentVolumeClaims. In GKE, common StorageClasses include `standard` backed by standard persistent disks and `premium-rwo` backed by SSDs. When a PVC references a StorageClass, GKE automatically provisions a PersistentVolume using the Google Cloud persistent disk CSI driver. For databases, I choose SSD-backed StorageClasses for better IOPS and latency. I can also set `volumeBindingMode: WaitForFirstConsumer` so the disk is provisioned in the same zone as the Pod that will use it, which is important for zonal persistent disks.

Q22. How do you handle node upgrades in GKE?

Answer: GKE supports automatic and manual node upgrades. For manual upgrades, I use `gcloud container clusters upgrade` or the Google Cloud Console. GKE creates new nodes with the new version, drains old nodes by evicting Pods gracefully using `PodDisruptionBudgets`, and then deletes the old nodes. To minimize disruption, I use regional clusters with nodes spread across zones, configure `PodDisruptionBudgets`, and ensure applications handle `SIGTERM` gracefully. For critical databases in `StatefulSets`, I perform upgrades carefully, one replica at a time, and verify cluster quorum before proceeding.

Q23. What is GKE's Cluster Autoscaler?

Answer: Cluster Autoscaler automatically adjusts the number of nodes in a node pool based on pending Pods. If Pods cannot schedule because of insufficient resources, the autoscaler adds nodes. If nodes are underutilized and Pods can be moved elsewhere, it removes nodes. It works with node pools and respects resource requests, taints, and affinity rules. For backend teams, this means applications scale without manual capacity planning. For DevOps engineers, it reduces cost by removing idle nodes, but it requires careful tuning to avoid thrashing and ensure scale-up happens fast enough for traffic spikes.

Q24. How do you implement network policies in GKE?

Answer: Network policies are Kubernetes resources that control traffic between Pods at the IP layer. In GKE, they require a CNI plugin that supports network policy enforcement, such as Calico. I define default-deny policies for ingress and egress in each namespace, then add allow policies for legitimate traffic such as from the ingress controller to the backend, from backend to database, and DNS egress. Network policies reduce the blast radius of a compromised Pod and enforce zero-trust communication within the cluster. They are essential for multi-tenant or regulated environments.

Scenario based questions

S1. You are asked to create a production-ready GKE cluster. What `gcloud` commands and flags do you consider?

Answer: I start by setting the project and region with `gcloud config set project <project>` and `gcloud config set compute/region <region>`. I create a regional cluster for high availability using `gcloud container clusters create <cluster> --region <region> --num-nodes 3 --enable-autoscaling --min-nodes 2 --max-nodes 10 --release-channel regular --enable-network-policy --enable-vertical-pod-autoscaling`. I enable Workload Identity with `--workload-pool=<project>.svc.id.goog` so Pods can use

GCP IAM without service account keys. I choose a private cluster with authorized networks for the control plane if security requires it. I then run `gcloud container clusters get-credentials <cluster> --region <region>` to configure kubectl. For storage, I verify the default StorageClass and create a premium SSD class if needed. Real-time example: a fintech company needs a GKE cluster in `us-central1`. I create a regional cluster with three zones, enable Workload Identity, Network Policy with Calico, cluster autoscaling, and node auto-repair. I restrict control plane access to corporate IP ranges. The cluster is ready for PCI-DSS scoped workloads after additional hardening.

S2. A GKE cluster needs access to Google Cloud Storage without using service account keys. How do you configure it?

Answer: I use GKE Workload Identity. First, I create a Google Cloud service account with the `roles/storage.objectViewer` role. I create a Kubernetes service account in the application namespace and annotate it with the Google service account email using `iam.gke.io/gcp-service-account`. I then bind the Kubernetes service account to the Google service account using `gcloud iam service-accounts add-iam-policy-binding` with the `roles/iam.workloadIdentityUser` role. The application uses the Google Cloud client library, which automatically obtains tokens through the metadata server. No JSON key is mounted in the Pod. Real-time example: an image processing service uploads files to Cloud Storage. Previously, a JSON key was mounted from a Secret and rotated quarterly. I migrate to Workload Identity, remove the key Secret, bind the Pod's service account to `storage-uploader@project.iam.gserviceaccount.com`, and the application uploads files using default credentials. Security improves and credential rotation is automatic.

S3. A Postgres database needs to run in GKE with persistent storage. How do you design the deployment?

Answer: I use a StatefulSet because Postgres needs stable identity and storage. I create a headless Service for stable DNS names per replica. I define a `volumeClaimTemplate` in the StatefulSet that requests storage from a premium SSD StorageClass, such as `premium-rwo`. I store credentials in a Kubernetes Secret and mount them as environment variables. I use a ConfigMap for `postgresql.conf` settings. I configure liveness and readiness probes, and an init container if replication setup is needed. I also set a PodDisruptionBudget with `maxUnavailable: 1` to protect quorum or availability during node drains. For production, I prefer Cloud SQL through a Private IP or Cloud SQL Auth Proxy unless the business requires running the database inside GKE. Real-time example: a SaaS product requires a self-managed Postgres cluster in GKE. I deploy a StatefulSet `postgres` with three replicas, each with a 100 GB premium-rwo PVC. The headless Service

`postgres` gives DNS names `postgres-0.postgres` , `postgres-1.postgres` , and `postgres-2.postgres` . I configure Patroni for leader election and failover. `PodDisruptionBudget` ensures only one replica is disrupted at a time during node maintenance.

S4. A microservice in namespace A cannot reach a service in namespace B. What do you check?

Answer: I verify the Service exists in namespace B and has endpoints with `kubectl get svc -n b` and `kubectl get endpoints -n b` . I check that the client is using the full DNS name `<service>.<namespace>.svc.cluster.local` . I inspect labels to ensure the Service selector matches the target Pods. I check NetworkPolicies in both namespaces to see if traffic is blocked. I run a debug Pod in namespace A and test connectivity with `curl` or `nslookup` . If DNS fails, I check CoreDNS Pods and ConfigMap. If NetworkPolicy is the issue, I add an allow rule from the source Pod selector to the destination port. Real-time example: the order service in namespace `orders` cannot call the payment service in namespace `payments` . The order service uses only `payment-service` without the namespace. I update the URL to `payment-service.payments.svc.cluster.local` . Additionally, a default-deny NetworkPolicy in `payments` blocks cross-namespace traffic, so I add an allow policy for Pods labeled `app: order-service` from namespace `orders` on port 8080.

S5. You need to expose a Spring Boot API on `api.shop.com` with HTTPS. What Kubernetes resources do you create?

Answer: I create a Deployment with the Spring Boot container, a Service of type ClusterIP selecting the Pods, and an Ingress that routes `api.shop.com` to the Service. I create a Kubernetes TLS Secret containing the certificate and key, and reference it in the Ingress `tls` section. I install or use the GKE Ingress Controller, which provisions a Google Cloud load balancer. I reserve a static global IP with `gcloud compute addresses create` and annotate the Ingress with `networking.gke.io/static-ip` . For DNS, I create an A record pointing `api.shop.com` to the static IP. I also configure backend health checks by ensuring the Spring Boot actuator health endpoint returns 200. Real-time example: an e-commerce API needs public HTTPS access. I deploy the service with a ClusterIP Service and an Ingress with host `api.shop.com` , TLS Secret `api-shop-tls` , and static IP annotation. After DNS propagation, external users reach the API over HTTPS, and Google Cloud Load Balancer terminates TLS.

S6. A developer accidentally commits a Kubernetes Secret manifest with a plaintext password to Git. What do you do?

Answer: First, I treat it as a security incident. I rotate the exposed credential immediately because the password is now in Git history and potentially exposed. I remove the Secret from the manifest and use a secret management solution such as Google Secret Manager, External Secrets Operator, or Sealed Secrets. If the password must be in a Kubernetes Secret, I generate it with `kubectl create secret generic` locally or in a secure CI step, and add the manifest to `.gitignore`. I also review Git history and consider rewriting history if the repository is private and small. Finally, I update the team runbook to forbid committing Secrets and add a CI check that rejects files containing `kind: Secret` with base64-looking values. Real-time example: a database password appears in `db-secret.yaml` pushed to GitHub. I rotate the database password, delete the manifest from the repo, configure External Secrets Operator to fetch the password from Google Secret Manager, and add a pre-commit hook that blocks Kubernetes Secret YAML files.

S7. A GitHub Actions workflow builds and deploys to GKE, but the deployment fails because kubectl cannot authenticate. How do you fix it?

Answer: I check that the workflow authenticates to Google Cloud using Workload Identity Federation or a service account key stored as a GitHub secret. I use `google-github-actions/auth` to authenticate, then `google-github-actions/get-gke-credentials` to configure kubectl. I verify the service account has `roles/container.developer` on the GKE cluster. I also confirm that the project ID, cluster name, and region in the workflow match the GKE cluster. If using Workload Identity Federation, I verify the GitHub repository and workflow identity mapping in the GCP workload identity pool provider. If the workflow uses a long-lived JSON key, I migrate to Workload Identity Federation for better security. Real-time example: a pipeline fails with `Unable to connect to the server: dial tcp ...: connect: connection refused`. The workflow used an old cluster name after a recent cluster recreation. I update the workflow to the new cluster name and region, configure Workload Identity Federation with the provider audience, and `get-gke-credentials` succeeds.

S8. You want to deploy a new image to GKE only after all tests pass. How do you structure the GitHub Actions workflow?

Answer: I split the workflow into jobs with dependencies. The first job checks out code, runs unit tests, integration tests, and code quality checks. The second job builds the Docker image and pushes it to

Artifact Registry with a tag such as the Git commit SHA. The third job deploys to a staging environment by updating the Kubernetes manifest and applying it or opening a pull request to the GitOps repository. The fourth job runs smoke tests against staging. A production deployment job requires manual approval using GitHub Environments and protection rules. Each job has `needs` dependencies so deployment only runs after earlier jobs succeed. This ensures no broken image reaches production. Real-time example: a Spring Boot service pipeline has `test`, `build`, `deploy-staging`, `smoke-tests`, and `deploy-prod` jobs. `deploy-prod` requires a manual approval from the release manager. The commit SHA image tag guarantees traceability from code to cluster.

S9. A GKE StatefulSet database loses data when a Pod is deleted. What is the likely cause?

Answer: The StatefulSet is not using persistent storage, or the PVC is not bound correctly. If the Pod uses an `emptyDir` volume, data is deleted when the Pod is removed. I check the StatefulSet manifest for `volumeClaimTemplates` and verify that each Pod has a bound PVC. I inspect the StorageClass and ensure it provisions volumes successfully. I also verify the reclaim policy; a `Delete` policy removes the PV when the PVC is deleted, which may be intentional but must be understood. The correct design is a `volumeClaimTemplate` so each ordinal Pod gets a stable PVC that is reattached on rescheduling. Real-time example: a Redis StatefulSet uses `emptyDir` for persistence. A node drains, the Pod moves, and all cache data is gone. I replace `emptyDir` with a `volumeClaimTemplate` using `premium-rwo`, and the new Pod mounts the same PVC after rescheduling, preserving data.

S10. You need to update a ConfigMap used by a running Deployment. Will the application pick up the change?

Answer: If the ConfigMap is mounted as a volume, Kubernetes updates the files in the running containers, but the application must watch or reload the files to use them. If the ConfigMap is injected as environment variables, the values are set at Pod creation time and will not update until the Pod restarts. To force a restart, I run `kubectl rollout restart deployment/<name>`. For applications that support hot reload, I implement a file watcher or periodic refresh. For most Java backend services, rolling restart is the simplest and most reliable method. Real-time example: a feature flag stored in a ConfigMap is updated, but the Spring Boot application still uses the old value. I mount the ConfigMap as a volume and add a `FileSystemWatcher` that refreshes the flag on change. I also keep `kubectl rollout restart` as a fallback for flags that are loaded once at startup.

S11. A GKE Ingress returns 502 Bad Gateway for a healthy service. What do you check?

Answer: A 502 from a Google Cloud load balancer usually means the backend Pods are unhealthy or the backend service configuration is wrong. I check the Service `targetPort` matches the container `containerPort`. I verify the readiness probe in the Deployment is correct and that Pods are marked ready. I check the Ingress backend health status in the Google Cloud Console. If the application binds to `127.0.0.1` inside the container, the load balancer health check cannot reach it, so I ensure the application listens on `0.0.0.0`. I also check firewall rules allowing health checks from Google Cloud IP ranges to the nodes. Real-time example: a Spring Boot application listens on `localhost:8080` for security reasons. The GCE health check targets the Pod IP and fails. I change the binding to `0.0.0.0:8080` and add a readiness probe on `/actuator/health/readiness`. The Ingress returns 200.

S12. A GKE cluster cost is too high. What optimizations do you consider?

Answer: I review node utilization with Cloud Monitoring and `kubectl top nodes`. I enable Cluster Autoscaler with appropriate min and max node counts to remove idle nodes. I use preemptible or spot instances for non-critical workloads. I right-size machine types based on actual usage. I use GKE Autopilot if workloads fit its constraints to pay per Pod. I implement Horizontal Pod Autoscaler to scale workloads based on demand. I schedule batch jobs during off-peak hours and use resource requests and limits to avoid over-provisioning. I also review LoadBalancer costs and consolidate external access through Ingress instead of multiple LoadBalancer Services. Real-time example: a staging cluster runs three `n2-standard-8` nodes 24/7 but is only used during business hours. I enable Cluster Autoscaler with min 0 and max 5, switch non-production workloads to spot instances, and add a scheduled scale-down script. Monthly cost drops by 60 percent.

S13. A database StatefulSet needs zero-downtime node maintenance. How do you proceed?

Answer: I configure a PodDisruptionBudget with `maxUnavailable: 1` so Kubernetes drains only one replica at a time. Before maintenance, I verify the database cluster is healthy and has quorum. I cordon the node to prevent new Pods from scheduling on it. I drain the node with `kubectl drain` which respects PodDisruptionBudgets. The StatefulSet reschedules the evicted Pod on another node and reattaches its PVC. I wait for the Pod to become ready and the cluster to synchronize before draining the next node. For single-replica databases, downtime is unavoidable unless a secondary is promoted. I also ensure the database handles SIGTERM gracefully to close connections cleanly. Real-

time example: A three-node MySQL InnoDB cluster runs on GKE. I set `maxUnavailable: 1` and use `kubectl drain` on each node sequentially. Each time, MySQL reschedules, reattaches its PVC, and rejoins the group. The application sees no downtime because the read-write Service always points to the current primary.

S14. A team wants GitOps with ArgoCD on GKE. How do you set it up?

Answer: I install ArgoCD in a dedicated namespace on the GKE cluster. I configure it to connect to the Git repository containing Kubernetes manifests or Helm charts. I create an ArgoCD Application for each microservice, pointing to the environment-specific path or branch. I enable auto-sync, self-heal, and prune so the cluster matches Git. I use ApplicationSet for multi-environment or multi-cluster deployments. I configure RBAC in ArgoCD to control who can sync or modify applications. Secrets are managed with Sealed Secrets or External Secrets Operator because ArgoCD should not read plaintext secrets from Git. I also configure webhooks or polling so changes are picked up quickly. Real-time example: a platform team adopts ArgoCD for GKE. They install ArgoCD via Helm, create an `app-of-apps` pattern, and configure each microservice Application to watch a GitOps repo. A developer merges a manifest change, ArgoCD detects the drift, syncs the cluster, and the Git commit history becomes the audit trail for all deployments.

S15. A developer asks how to debug a failing Pod in GKE. What commands and tools do you recommend?

Answer: I recommend a systematic approach. First, `kubectl get pods -n <ns>` to see the status. Second, `kubectl describe pod <pod> -n <ns>` to read events. Third, `kubectl logs <pod> -n <ns>` and `kubectl logs --previous` for crash forensics. Fourth, `kubectl exec -it <pod> -- /bin/sh` to inspect the running container if it is still alive. If the image is distroless, I use a debug ephemeral container with `kubectl debug`. For networking issues, I run a debug Pod with `nicolaka/netshoot` and test DNS, routing, and connectivity. I also check Google Cloud Logging and Monitoring for cluster-level events and resource saturation. Real-time example: a Pod shows `CrashLoopBackOff`. `kubectl logs --previous` shows a missing table error. I exec into the database Pod, confirm the migration did not run, check the init container logs, and find that the migration container used the wrong database host. I fix the ConfigMap and the application starts.

S16. A GKE Pod needs a GPU. How do you configure the node pool and workload?

Answer: I create a node pool with GPU-enabled machine types using `gcloud container node-pools create <pool> --cluster <cluster> --accelerator type=<gpu-type>,count=<count>`. I install the NVIDIA driver daemonset, which GKE can deploy automatically with `--enable-autoprovisioning` or manually. I add a nodeSelector or node affinity to the Pod to schedule it on GPU nodes, such as `cloud.google.com/gke-accelerator: nvidia-tesla-t4`. I set resource limits for `nvidia.com/gpu` in the Pod spec. I also consider taints and tolerations to prevent non-GPU workloads from scheduling on expensive GPU nodes. Real-time example: a machine learning inference service needs a T4 GPU. I create a GPU node pool with autoscaling from 0 to 2 nodes, add a toleration and nodeSelector to the inference Deployment, and set `resources.limits.nvidia.com/gpu: 1`. The Pod schedules only when GPU capacity is available, and the node scales to zero when no inference traffic is running.

S17. A new release causes errors in production. How do you roll back in GKE?

Answer: If the release was a Deployment update, I run `kubectl rollout undo deployment/<name> -n <ns>` to revert to the previous revision. I verify the rollback with `kubectl rollout status` and monitor error rates and latency. If the release was applied through GitOps, I revert the manifest change in Git and let ArgoCD sync the cluster to the previous state. For critical services, I use a blue-green or canary strategy so I can switch traffic back to the stable version quickly without waiting for a full Pod restart. After rollback, I analyze logs and metrics to identify the root cause before attempting the release again. Real-time example: a new version of the payment service has 5 percent error rate. I run `kubectl rollout undo deployment/payment-service -n prod`, which restores the previous ReplicaSet. Error rate returns to baseline within two minutes. I then inspect the new version's logs and find an incompatible database migration that ran in staging but not with the previous prod schema.

S18. A GKE cluster must comply with a no-public-IP policy for control plane access. How do you configure it?

Answer: I create a private cluster with a private control plane endpoint using `gcloud container clusters create <cluster> --enable-private-nodes --enable-private-endpoint --master-ipv4-cidr <cidr>`. With a private endpoint, the control plane is accessible only from authorized networks or through a bastion host or Cloud Interconnect. I add authorized networks for the CI/CD runner IP ranges and the corporate VPN. I also ensure nodes have no external IPs by using `--create-subnetwork` with private nodes. For workloads that need outbound internet access, I configure Cloud NAT. This setup is common in regulated industries and prevents direct internet

exposure of the Kubernetes API server. Real-time example: a healthcare company requires a private GKE cluster. I create a private cluster with a `/28` master CIDR, add authorized networks for the office and CI runner, configure Cloud NAT for egress, and restrict worker nodes to private IPs only. Security audits pass without public API exposure.

S19. A microservice needs environment-specific feature flags. How do you manage them in GKE?

Answer: I store feature flags in a ConfigMap per namespace, one for each environment. The Deployment mounts the ConfigMap as a file or injects values as environment variables. The application reads the flag value and toggles behavior accordingly. For more advanced use cases, I use a feature flag service such as LaunchDarkly, Split, or a custom flag API. When a flag changes, I either update the ConfigMap and restart the Deployment, or use a flags API that supports dynamic updates. ConfigMaps work well for static per-environment flags, while a flag service is better for user-targeted or gradual rollouts. Real-time example: an e-commerce service needs a new checkout UI in staging but not production. I add `feature.new-checkout=true` to the staging ConfigMap and `feature.new-checkout=false` to the production ConfigMap. The same image runs in both environments, and behavior changes based on the mounted configuration.

S20. A GKE workload needs to handle traffic spikes automatically. How do you set up autoscaling?

Answer: I configure Horizontal Pod Autoscaler on the Deployment using CPU or custom metrics. I install the GKE Metrics Server if not already available. The HPA scales replicas between a configured minimum and maximum based on target utilization. I also enable Cluster Autoscaler on the node pool so new nodes are added when HPA creates Pods that cannot fit. For rapid spikes, I set lower target CPU and buffer capacity with `maxSurge`. For event-driven workloads, I use Cloud Run or Google Cloud Pub/Sub with a Kubernetes consumer autoscaled on queue depth. I verify autoscaling behavior with load tests. Real-time example: a ticket booking service sees tenfold traffic spikes during sales. I add an HPA targeting 60 percent CPU with `minReplicas: 3` and `maxReplicas: 50`. I enable Cluster Autoscaler on the node pool with max 20 nodes. Load testing shows Pods scale up within 60 seconds and nodes scale up within two minutes, handling spikes without manual intervention.